

PROJECT AND TEAM INFORMATION

Project Title

OSIRIS-OPERATING SYSTEM INCIDENT RESPONSE & INTELLIGENCE SYSTEM

Student / Team Information

<i>TEAM NAME</i>	
<i>TEAM MEMBER 1 (TEAM LEADER)</i>	Divyanshu - 240111928 divyanshsir2005@gmail.com 
<i>TEAM MEMBER 2</i>	Jahanvi jaiswal - 240221839 Jaiswaljahanvi01@gmail.com 
<i>TEAM MEMBER 3</i>	Rudransh Devli - 240211286 rudranshdevli06@gmail.com 

PROPOSAL DESCRIPTION

Motivation

Modern operating systems continuously generate large amounts of information related to processes, users, files, system resources, and system events. During a security incident, this information may be distributed across different system interfaces and logs, making it difficult to understand what happened, when it happened, and which activities were related.

OSIRIS (Operating System Incident Reconstruction and Intelligence System) aims to provide a unified Linux-based platform for monitoring and investigating system activity. The system will collect selected operating-system activities, convert them into structured events, and store them with relevant information such as timestamps, processes, users, files, actions, and event sources.

The project is motivated by the need to move beyond simple real-time monitoring. Instead of only showing the current state of a system, OSIRIS will maintain a historical record of system activity. This will allow an investigator to examine a particular time period, reconstruct relevant system activity, identify suspicious patterns, and understand the sequence of events associated with a security incident.

State of the Art / Current solution

Linux provides several mechanisms for monitoring operating-system activity, including process information, system logs, filesystem events, resource statistics, and auditing facilities. Existing monitoring and security tools can collect such information and generate alerts, while enterprise security platforms can provide centralized event analysis.

However, these mechanisms are often used through separate tools, with process information, file activity, system logs, and security alerts being examined independently. This can make incident investigation dependent on manually correlating information from multiple sources.

OSIRIS proposes a unified approach in which selected Linux activities are collected, converted into a common event format, and stored in a relational database. The stored information maintains relationships among system entities such as users, processes, files, resources, events, alerts, and incidents. Our proposed system is not intended to replace enterprise monitoring or security products. Instead, it focuses on developing prototype that demonstrates how operating-system activity can be continuously collected, historically preserved, queried, reconstructed, and analyzed for security purposes.

Project Goals and Milestones

The primary goal of OSIRIS is to develop a Linux-based platform capable of monitoring system activity, maintaining historical records, reconstructing system behavior, detecting potentially suspicious activities, and supporting security incident investigation.

Phase 1: Observe & Collect

The first phase will establish the monitoring and data-management foundation of OSIRIS. Selected Linux activities will be collected and the information will be converted into a standardized event format and stored in a relational database.

Milestones:

- Linux monitoring environment setup
- Database and ER-model design
- Process monitoring
- Filesystem monitoring
- System-resource monitoring
- Event normalization
- Relational database implementation
- Backend API
- Monitoring dashboard
- Initial system testing

Phase 2: Reconstruct & Detect

The second phase will transform the monitoring platform into a security investigation system. Historical events and selected system-state information will be used to reconstruct system activity over a specified time period. A security-analysis component will apply predefined and explainable rules to identify potentially suspicious activities.

Milestones:

- Historical event timeline
- System activity reconstruction
- Security detection rules
- File-integrity monitoring
- Alert generation
- Incident management
- Evidence-based investigation
- DBMS transaction and concurrency demonstrations
- Final testing and evaluation

Phase 3: Analyze and Polish

Evaluate the complete system, improve accuracy and performance, integrate all modules, fix issues, strengthen security, and prepare the final working product.

Project Approach

Modular and Incremental Development:

- Develop OSIRIS in a controlled Linux environment, preferably a virtual machine.
- Separate the system into modules for data collection, event processing, database storage, security analysis, and visualization.

Linux Activity Monitoring:

- Implement Linux-specific collectors for process, file system, and system-resource monitoring.
- Collect process information such as process ID, parent process, user, process state, and selected resource information.
- Monitor configured directories for file creation, modification, deletion, and renaming events.
- Periodically collect CPU, memory, disk, and other selected system information.

Event Processing and Normalization:

- Convert information collected from different monitoring components into a common event format.
- Associate events with relevant entities such as users, processes, files, and system hosts.
- Preserve timestamps and event context to support later correlation, reconstruction, and investigation.

Relational Database Implementation:

- Store collected events in a PostgreSQL relational database.
- Design the database using ER modeling and normalization.
- Implement primary keys, foreign keys, integrity constraints, and appropriate relationships between entities.
- Use SQL queries, indexing, views, triggers, transactions, and concurrency control to support both project functionality and DBMS concepts.

Historical Activity Reconstruction:

- Maintain historical records of system activity rather than relying only on the current system state.
- Retrieve and correlate events from a selected time period.
- Use stored events and selected system-state information to reconstruct relevant system activity.
- Present reconstructed activity in a chronological timeline for investigation.

Alert and Incident Management:

- Generate alerts when defined security rules identify potentially suspicious activities.
- Organize related alerts into incidents.
- Maintain relevant incident information for investigation and tracking.
- Allow investigators to examine related timelines, users, processes, files, events, and supporting evidence.

Investigation and Visualization:

- Provide a dashboard for viewing collected system activity and security alerts.
- Provide investigation-oriented views for examining historical events and reconstructed activity.
- Present relevant evidence in a structured form to support analysis of an incident.

Validation and Reliability:

- Test the individual monitoring, processing, storage, detection, and investigation components.
- Use controlled system activities to verify that events are correctly collected, stored, reconstructed, and detected.
- Validate database integrity and investigate query performance using appropriate indexes and query-analysis techniques.

Project Outcome / Deliverables

- **OSIRIS Prototype:** A functional Linux-based platform for monitoring system activities and supporting security investigation.
- **Activity Monitoring:** Collection of selected process, user, file system, and resource activities using Linux system interfaces.
- **Event Management:** Conversion of collected activities into structured, timestamped events with relevant contextual information.
- **PostgreSQL Database:** A normalized relational database with ER modeling, keys, constraints, SQL queries, indexing, views, triggers, transactions, and concurrency control.
- **Historical Reconstruction:** Timeline-based examination and reconstruction of system activity during a selected period.
- **Security Detection:** Explainable rule-based identification of suspicious activities such as authentication failures, unusual process behavior, privilege-related activity, and protected-file modifications.
- **File Integrity Monitoring:** Detection of unexpected changes to selected protected files or directories.
- **Alert and Incident Management:** Generation of alerts, grouping of related events into incidents, and examination of supporting evidence.
- **Testing and Validation:** Test results covering monitoring, database operations, detection rules, reconstruction, alerting, and system integration.
- **Documentation:** System architecture, database schema, event structure, detection rules, installation, configuration, and usage documentation.
- **Reproducible Environment:** Controlled Linux/VM setup for demonstrating the complete OSIRIS workflow.

System Architecture:

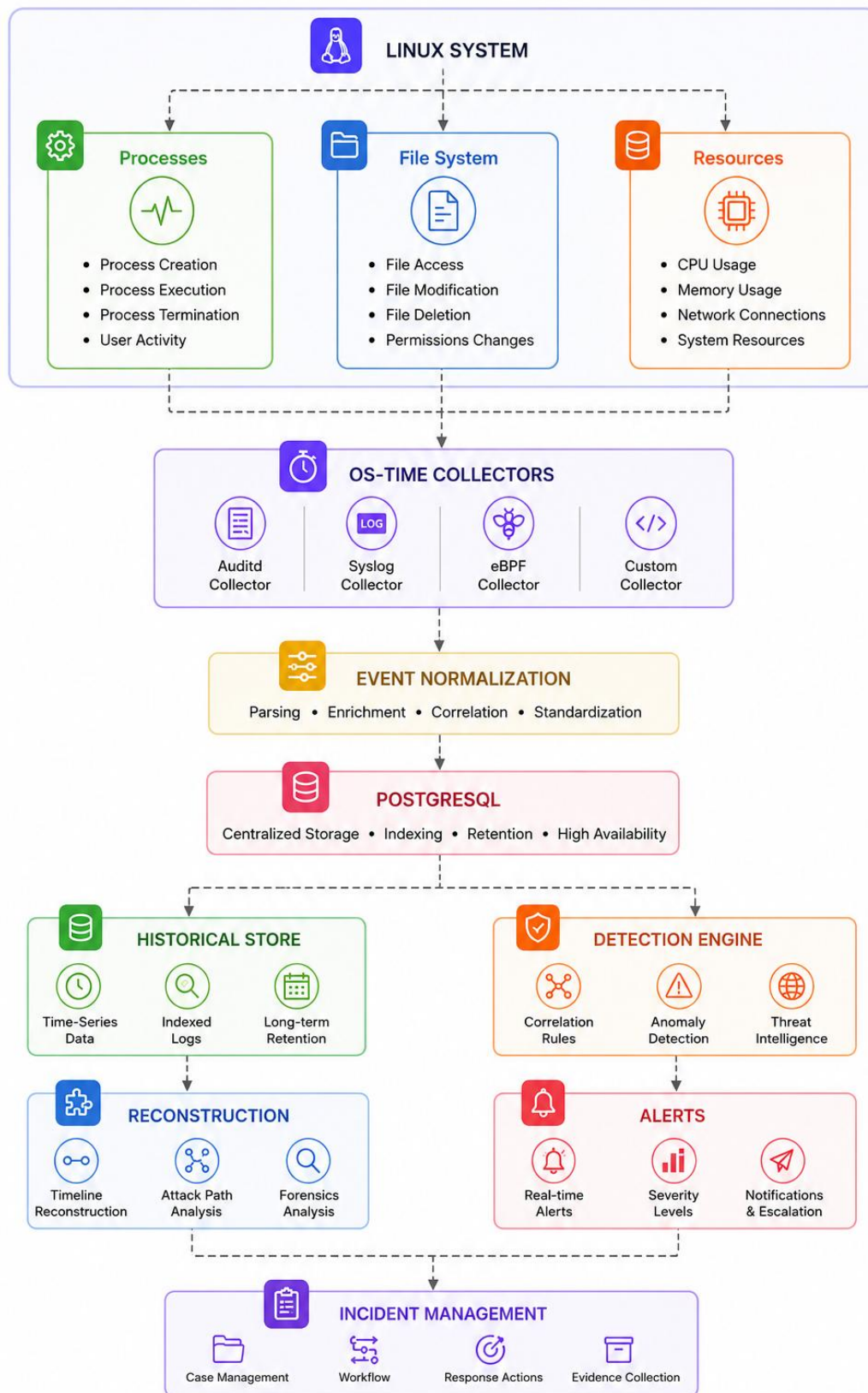


Figure 1: Linux Security Monitoring Architecture

Assumptions

The system will initially be developed and tested on a Linux virtual machine in a controlled environment. Only selected operating-system activities and configured filesystem paths will be monitored. Required Linux interfaces and permissions are assumed to be available to the monitoring components. Synthetic and controlled test scenarios will be used instead of monitoring private or unauthorized systems. Security rules will identify potentially suspicious behavior but will not automatically prove malicious intent. Historical reconstruction will be limited to information successfully collected and stored by the system.

References

BOOKS:

- A. Silberschatz, P. B. Galvin, and G. Gagne, Operating System Concepts, 10th ed., Wiley, 2018/2019.
- R. ElmaNavathe S. B. Navathe, Fundamentals of Database Systems, 7th ed., Pearson, 2015/2017.
- J. F. Kurose and K. W. Ross, Computer Networking: A Top-Down Approach, 8th ed., Pearson, 2022.

LINKS:

- E. Bärtil, “cortex-xdr-client: A Python-based API client for Cortex XDR API,” GitHub repository. <https://github.com/ebarti/cortex-xdr-client>
- The Linux Kernel Documentation, “The /proc Filesystem.” <https://www.kernel.org/doc/html/latest/filesystems/proc.html>
- PostgreSQL Global Development Group, “PostgreSQL 18 Documentation.” <https://www.postgresql.org/docs/current/>

RESEARCH PAPERS:

- S. Ma, J. Zhai, Y. Kwon, K. H. Lee, X. Zhang, G. Ciocarlie, A. Gehani, V. Yegneswaran, D. Xu, and S. Jha, “Kernel-supported cost-effective audit logging for causality tracking,” in 2018 USENIX Annual Technical Conference (USENIX ATC 18), Boston, MA, USA, 2018, pp. 241–254. <https://www.usenix.org/conference/atc18/presentation/ma-shiqing>
- S. Ma, J. Zhai, F. Wang, K. H. Lee, X. Zhang, and D. Xu, “MPI: Multiple perspective attack investigation with semantic aware execution partitioning,” in 26th USENIX Security Symposium (USENIX Security 17), Vancouver, BC, Canada, 2017, pp. 1111–1128. <https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/ma>